DNS-Based Traffic Steering for IPv6-Enabled Edge Microservices

Extended Project Description

Joar Heimonen
contact@joar.me

May 21, 2026

Contents

1 Introduction

The modern cloud computing stack relies heavily on Layer 7 proxies and service meshes to manage traffic between services however this comes at a cost. These layer 7 proxies introduce single points of failure in otherwise distributed systems, whilst adding unneccesary complexity and reducing the effectiveness of layer 3 routing. Service meshes add sidecar containers to every pod. This overhead scales with the amount of microservices you have.

This architectural pattern did not emerge by design. IPv4 exhaustion forced the development of technologies like NAT (Network Address Translation) and shared address spaces. With the introduction of NAT end-to-end accessibility was broken making proxies a structural necessity.

With a rapid increase in IPv6 adoption it is time to leave the technical limitations of the past in the past and build our services with IPv6s large adress space in mind. Under RIPE-738 [1], a Local internet registry recieves a minimum allocation size of CIDR /32 up to /29 without needing to justify it. A /29 allocation alone contains around 147 billion times more addresses than the whole IPv4 address space.

This thesis investigates whether DNS-based traffic steering, combined with direct IPv6 addressing of microservices can serve as a viable alternative to traditional layer 7 ingress proxies and service meshes. Rather than routing traffic trough a centralized proxy each pod is assigned a globally routable address and made directly reachable by clients. Traffic steering decissions are delegated to a purpose built DNS server that dynamically manages record based on pod health and metrics.

The remainder of this document surveys the state of the art in traffic steering and proxy scalability, identifies the gap that motivates this research and outlines the proposed approach, experimental methodology and work plan.

2 State of the Art in Traffic Steering and Proxy Scalability

2.1 The End-to-End Principle and the Role of NAT

Saltzer, Reed and Clark established in 1984 that application specific functions should be implemented at the endpoints of a communication system rather than within the network itself [2]. A network that simply moves packets without enforcing control points is inherently more ressilient. There are no forced bottlenecks, no single points of failure introduces by the infrastructure itself. Packets are free to be routed as the network sees fit.

The widespread adoption of NAT under IPv4 fundementally broke this principle. By placing address translation logic inside the network, NAT made end-to-end addressability impossible. Applications could no longer communicate directly over the internet. They required proxies, STUN servers and other middleboxes to facilitate communication compansating for the loss of globally unique addressing.

IPv6 restores the conditions under which end-to-end addressability principle can be applied. With a globally unique address assigned to every endpoint there is no longer a structural requirement for address translation or centralized proxies. The network can return to its intended role, moving packets between endpoints that are directly reachable by design.

2.2 Layer 7 Proxies and Ingress Controllers

2.3 Service Mesh Architectures

2.4 Kernel and Hardware Acceleration

2.5 DNS-Based Traffic Steering

2.6 Edge and Orchestration Context

3 Literature Search Methodology

The literature search was conducted using Google Scholar and Google Scholar Labs. Search terms were initially generated with the assistance of a large language model based on the research topic, and subsequently refined through iterative searching. The following keyword queries were used:

- *DNS TTL load balancing consistency tradeoff*
- *IPv6 end-to-end addressing NAT elimination*
- *eBPF Kubernetes networking performance*
- *service mesh overhead scalability microservices*
- *DNS-based server selection load balancing*

Papers were selected based on relevance to the research problem, publication venue quality, and citation count. The search was supplemented by backward snowballing from the reference lists of key papers, through which several foundational works were identified. Papers published in predatory or low-quality journals were excluded.

The resulting set of nine papers is summarized in Table 1.

#	Authors	Year	Key Claim
1	Wang, Shou, Qian, Liu	2026	In-kernel eBPF L7 load balancer, 1.5x throughput over Istio [3]
2	Li, Lu, Lin, Wu, Zhang, Yan	2023	DPU-centric service mesh, 90% latency reduction vs Envoy [4]
3	Hawley	2009	GeoDNS geographic traffic steering at Internet scale [5]
4	Jeffery, Howard, Mortier	2021	etcd strong consistency bottleneck in edge Kubernetes [6]
5	Miziński, Przyłucki	2025	Cilium eBPF vs iptables: lower CPU/memory under load [7]
6	Moura, Heidemann, Schmidt	2019	DNS TTL tradeoffs between caching efficiency and routing flexibility [8]
7	Shaikh, Tewari, Agrawal	2001	Reducing DNS TTL increases nameserver load by orders of magnitude [9]
8	Chatterjee, Tari, Zomaya	2005	Adaptive TTL approach reduces client-perceived latency by 16% [10]
9	Saltzer, Reed, Clark	1984	End-to-end principle: functions belong at endpoints, not in the network [2]

Table 1: Papers included in the literature review

4 Problem Description

5 Discussion

6 Proposed Approach and Methodology

7 Work Plan and Milestones

Spring 2026

- Systematization of knowledge and literature study
- Write and submit mandatory essay on the state of proxy scalability
- Restructure essay into draft systematization of knowledge chapter
- Start development of DNS server

Autumn 2026

- Finish development of DNS server and custom ingress controller
- Implement client simulator and dummy microservices
- Select baseline paradigms
- Start developing the experiments
- Begin writing thesis background and methodology chapters

Spring 2027

- Perform the experiments
- Analyze the results
- Write article on DNS-based traffic steering
- Finish master thesis

8 Conclusion

References

[1] *IPv6 Address Allocation and Assignment Policy*, https://www.ripe.net/publications/docs/ripe-738/. Accessed: May 21, 2026.

[2] J. H. Saltzer, D. P. Reed, and D. D. Clark, "End-to-end arguments in system design," *ACM Transactions on Computer Systems*, vol. 2, no. 4, pp. 277–288, Nov. 1984, ISSN: 0734-2071, 1557-7333. DOI: 10.1145/357401.357402 Accessed: May 21, 2026.

[3] Y. Wang, C. Shou, J. Qian, and G. Liu, *XLB: A High Performance Layer-7 Load Balancer for Microservices using eBPF-based In-kernel Interposition*, 2026. DOI: 10.48550/ARXIV.2602.09473 Accessed: May 21, 2026.

[4] M. Li, W. Lu, H. Lin, J. Wu, Y. Zhang, and G. Yan, "FlatProxy: A DPU-centric Service Mesh Architecture for Hyperscale Cloud-native Application," vol. 14, no. 8, 2023.

[5] J. Hawley, "GeoDNS—Geographically-aware, protocol-agnostic load balancing at the DNS level,"

[6] A. Jeffery, H. Howard, and R. Mortier, "Rearchitecting Kubernetes for the Edge," in *Proceedings of the 4th International Workshop on Edge Systems, Analytics and Networking*, ser. EdgeSys '21, New York, NY, USA: Association for Computing Machinery, Apr. 2021, pp. 7–12, ISBN: 978-1-4503-8291-5. DOI: 10.1145/3434770.3459730 Accessed: May 21, 2026.

[7] K. Miziński and S. Przyłucki, "The impact of using eBPF technology on the performance of networking solutions in a Kubernetes cluster," *Journal of Computer Sciences Institute*, vol. 35, pp. 150–158, Jun. 2025, ISSN: 2544-0764. DOI: 10.35784/jcsi.7110 Accessed: May 21, 2026.

[8] G. C. M. Moura, J. Heidemann, R. d. O. Schmidt, and W. Hardaker, "Cache Me If You Can: Effects of DNS Time-to-Live," in *Proceedings of the Internet Measurement Conference*, ser. IMC '19, New York, NY, USA: Association for Computing Machinery, Oct. 2019, pp. 101–115, ISBN: 978-1-4503-6948-0. DOI: 10.1145/3355369.3355568 Accessed: May 21, 2026.

[9] A. Shaikh, R. Tewari, and M. Agrawal, "On the effectiveness of DNS-based server selection," in *Proceedings IEEE INFOCOM 2001. Conference on Computer Communications. Twentieth Annual Joint Conference of the IEEE Computer and Communications Society (Cat. No.01CH37213)*, vol. 3, Apr. 2001, 1801–1810 vol.3. DOI: 10.1109/INFCOM.2001.916678 Accessed: May 21, 2026.

[10] D. Chatterjee, Z. Tari, and A. Zomaya, "A task-based adaptive TTL approach for Web server load balancing," in *10th IEEE Symposium on Computers and Communications (ISCC'05)*, Jun. 2005, pp. 877–884. DOI: 10.1109/ISCC.2005.19 Accessed: May 21, 2026.